CHULETA DEFINITIVA EXAMEN BACKEND - DELIVERUS

Plantillas de Copiar, Pegar y Adaptar orientadas a la arquitectura de la asignatura. Optimizada a prueba de fallos.

CHECKLIST DE SUPERVIVENCIA (Antes de escribir código)

1. **El Diagrama/Tests MANDAN:** Si piden que el creador se llame `customerId` o `userId`, úsalo a rajatabla en TODA la app (Migraciones, Modelos, Controladores y Middlewares). Ante la duda, mira el seeder del profesor.
2. **Si creas un Modelo nuevo, actualiza al padre:** Si una Review pertenece a un user a través de `customerId`, tienes que ir a `User.js` y poner `User.hasMany(models.Review, { foreignKey: 'customerId' })`. Si no, Sequelize crashea con Error 500.
3. **Cuidado con las URLs en los Middlewares:** Si tu ruta es `/restaurants/:restaurantId`, en `req.params` SOLO existe `restaurantId`. Si intentas buscar `req.params.orderId`, será `undefined` y petará.
4. **Fechas obligatorias:** Nunca olvides `createdAt` y `updatedAt` en las migraciones o dará error 500.

---

1. MIGRACIONES (`database/migrations/...-create-tumodelo.js`)

REGLAS DE ORO: Nombre de la tabla en **PLURAL** (ej: Reviews, Schedules). No olvides NUNCA `createdAt` y `updatedAt`.

```
module.exports = {
  up: async (queryInterface, Sequelize) => {
    await queryInterface.createTable('TuModelos', {
      id: {
        allowNull: false,
        autoIncrement: true,
        primaryKey: true,
        type: Sequelize.INTEGER
      },
      name: { allowNull: false, type: Sequelize.STRING },
      price: { allowNull: false, defaultValue: 0.0, type: Sequelize.DOUBLE },

      // FORÁNEA 1 (Padre de la ruta, ej: Restaurant)
      restaurantId: {
        allowNull: false,
        type: Sequelize.INTEGER,
        onDelete: 'CASCADE', // Solo si piden que al borrar el restaurante se borre esto
```

```
        references: { model: { tableName: 'Restaurants' }, key: 'id' } // Nombre en PLURAL
      },

      // FORÁNEA 2 (El usuario dueño, mirar bien si es userId o customerId en el examen)
      customerId: {
        allowNull: false,
        type: Sequelize.INTEGER,
        onDelete: 'CASCADE',
        references: { model: { tableName: 'Users' }, key: 'id' }
      },

      // ¡NUNCA OLVIDAR LAS FECHAS!
      createdAt: { allowNull: false, type: Sequelize.DATE, defaultValue: new Date() },
      updatedAt: { allowNull: false, type: Sequelize.DATE, defaultValue: new Date() }
    })
  },
  down: async (queryInterface, Sequelize) => {
    await queryInterface.dropTable('TuModelos')
  }
}
```

---

2. MODELO (`models/TuModelo.js`)

REGLA DE ORO: Nombre en **SINGULAR**. La foreignKey debe ser exactamente igual a la que pusiste en la migración.

```
import { Model } from 'sequelize'

const loadModel = (sequelize, DataTypes) => {
  class TuModelo extends Model {
    static associate (models) {
      // Relaciones BelongsTo (Hijo -> Padre)
      TuModelo.belongsTo(models.Restaurant, { foreignKey: 'restaurantId', as: 'restaurant' })
      TuModelo.belongsTo(models.User, { foreignKey: 'customerId', as: 'user' }) // OJO AL NOM
    }
  }

  TuModelo.init({
    name: { allowNull: false, type: DataTypes.STRING },
    price: { allowNull: false, type: DataTypes.DOUBLE },
    restaurantId: { allowNull: false, type: DataTypes.INTEGER },
    customerId: { allowNull: false, type: DataTypes.INTEGER }, // OJO AL NOMBRE
    createdAt: { allowNull: false, type: DataTypes.DATE, defaultValue: new Date() },
    updatedAt: { allowNull: false, type: DataTypes.DATE, defaultValue: new Date() }
  }, {
    sequelize,
    modelName: 'TuModelo'
  })
  return TuModelo
```

```
}
export default loadModel
```

---

3. CAMPOS VIRTUALES (Cálculos de Medias o Totales)

Se añaden dentro de un modelo padre (ej: `Restaurant.js`) para calcular datos de sus hijos. **REGLA DE ORO:** Siempre comprueba si el array está vacío o devolverás NaN (División por cero) y el test de "empty array" fallará.

```
class Restaurant extends Model {
  // PASO 1: LA FUNCIÓN MATEMÁTICA (Dentro de la clase, bajo associate)
  async getAvgStars () {
    try {
      const elementos = await this.getTuModelos() // Ej: this.getReviews()
      // VITAL: Si no hay elementos, devolver null o 0 según pida el test
      if (!elementos || elementos.length === 0) return null

      const suma = elementos.reduce((acc, item) => acc + item.stars, 0)
      return suma / elementos.length
    } catch (err) {
      return err
    }
  }
}

Restaurant.init({
  // ... tus campos normales

  // PASO 2: EL CAMPO VIRTUAL
  avgStars: {
    type: DataTypes.VIRTUAL,
    get () { return this.getDataValue('avgStars') },
    set (value) { this.setDataValue('avgStars', value) }
  }
}, {
  sequelize,
  modelName: 'Restaurant',

  // PASO 3: EL HOOK DISPARADOR (Se calcula solo al hacer un GET)
  hooks: {
    afterFind: async (result) => {
      if (Array.isArray(result)) {
        await Promise.all(result.map(async item => {
          item.dataValues.avgStars = await item.getAvgStars()
        }))
      } else if (result) {
        result.dataValues.avgStars = await result.getAvgStars()
      }
    }
```

```
  }
})
```

---

4. VALIDACIONES BÁSICAS

(`controllers/validation/TuModeloValidation.js`)

REGLAS DE ORO:

1. Escribir `exists()` (con 'S'). Si pones `exist()` la app explota al arrancar (Error 404 global).
2. Si un número puede ser 0 (ej: 0 estrellas permitidas), usa `min: 0`. Si pones `min: 1`, dará Error 422.

```
import { check } from 'express-validator'

const create = [
  check('name').exists().isString().isLength({ min: 1, max: 255 }).trim(),
  // Campo Opcional (Si el test manda peticiones donde falta este campo)
  check('body').optional({ nullable: true, checkFalsy: true }).isString().trim(),
  // Campo numérico (Cuidado con el minimo permitido por el test, ej: 0)
  check('stars').exists().isInt({ min: 0, max: 5 }).toInt(),

  // Prohibir inyección manual de IDs por seguridad
  check('restaurantId').not().exists(),
  check('customerId').not().exists(), // O userId
  check('id').not().exists()
]

const update = [ ...create ]

export { create, update }
```

---

5. MIDDLEWARES (`middlewares/TuModeloMiddleware.js`)

REGLAS DE ORO: Construye los `where` usando solo `req.user.id` y los parámetros de la URL actual (ej: `req.params.restaurantId`).

```
import { TuModelo, Order } from '../models/models.js'

// EJEMPLO A: REQUISITO PREVIO (Ej: Tener pedidos antes de valorar) -> Devuelve 409
const checkUserHasOrders = async (req, res, next) => {
  try {
    const count = await Order.count({
      where: {
        userId: req.user.id, // OJO: Order en DeliverUS siempre usa userId
        restaurantId: req.params.restaurantId
      }
```

```
    })
    if (count > 0) return next() // Si tiene 1 o más pedidos, pasa
    return res.status(409).send('User has no orders.')
  } catch (err) { return res.status(500).send(err.message) }
}

// EJEMPLO B: EVITAR DUPLICADOS (Ej: Solo 1 reseña por restaurante) -> Devuelve 409
const checkNotDuplicated = async (req, res, next) => {
  try {
    const count = await TuModelo.count({
      where: {
        customerId: req.user.id, // La clave foránea de tu modelo
        restaurantId: req.params.restaurantId
      }
    })
    if (count === 0) return next() // Si tiene 0, le dejamos crearla
    return res.status(409).send('Entity already exists.')
  } catch (err) { return res.status(500).send(err.message) }
}

// EJEMPLO C: COMPROBAR PROPIEDAD (Evitar que modifique algo de otro) -> Devuelve 403
const checkOwnership = async (req, res, next) => {
  try {
    const item = await TuModelo.findByPk(req.params.tuModeloId)
    if (item.customerId !== req.user.id) { // Comparamos dueño vs logueado
      return res.status(403).json({ message: 'Not enough privileges.' })
    }
    next()
  } catch (err) { return res.status(500).send(err.message) }
}

// EJEMPLO D: COMPROBAR DEPENDENCIA (No borrar si tiene hijos) -> Devuelve 409
const checkHasNoOrders = async (req, res, next) => {
  try {
    const numberOfOrders = await Order.count({
      where: { tuModeloId: req.params.tuModeloId }
    })
    if (numberOfOrders === 0) return next() // Solo borramos si no tiene hijos
    return res.status(409).send('Some orders belong to this entity.')
  } catch (err) { return res.status(500).send(err.message) }
}

export { checkUserHasOrders, checkNotDuplicated, checkOwnership, checkHasNoOrders }
```

---

6. CONTROLADOR (`controllers/TuModeloController.js`)

```
import { TuModelo } from '../models/models.js'

const TuModeloController = {
  async index (req, res) {
    try {
```

```
      const items = await TuModelo.findAll({ where: { restaurantId: req.params.restaurantId }
      res.json(items)
    } catch (err) { res.status(500).send(err) }
  },

  async show (req, res) {
    try {
      const item = await TuModelo.findByPk(req.params.tuModeloId)
      res.json(item)
    } catch (err) { res.status(500).send(err) }
  },

  async create (req, res) {
    try {
      const newItem = TuModelo.build(req.body)
      // 1. Inyectamos ID de la URL
      if (req.params.restaurantId) newItem.restaurantId = req.params.restaurantId
      // 2. Inyectamos el ID del usuario logueado
      newItem.customerId = req.user.id // O userId, según corresponda

      const savedItem = await newItem.save()
      res.json(savedItem)
    } catch (err) { res.status(500).send(err) }
  },

  async update (req, res) {
    try {
      await TuModelo.update(req.body, { where: { id: req.params.tuModeloId } })
      const updatedItem = await TuModelo.findByPk(req.params.tuModeloId)
      res.json(updatedItem)
    } catch (err) { res.status(500).send(err) }
  },

  async destroy (req, res) {
    try {
      const result = await TuModelo.destroy({ where: { id: req.params.tuModeloId } })
      let message = result === 1 ? 'Successfully deleted.' : 'Could not delete.'
      res.json(message)
    } catch (err) { res.status(500).send(err) }
  }
}
export default TuModeloController
```

---

7. RUTAS (`routes/TuModeloRoutes.js`)

Orden de ejecución vital: Autenticación -> Existe Padre -> Existe Elemento -> Middlewares Negocio -> Validaciones -> Controlador.

```
import TuModeloController from '../controllers/TuModeloController.js'
import * as TuModeloValidation from '../controllers/validation/TuModeloValidation.js'
```

```
import { isLoggedIn, hasRole } from '../middlewares/AuthMiddleware.js'
import { handleValidation } from '../middlewares/ValidationHandlingMiddleware.js'
import { checkEntityExists } from '../middlewares/EntityMiddleware.js'
import { Restaurant, TuModelo } from '../models/models.js'
import * as Mid from '../middlewares/TuModeloMiddleware.js'

const loadRoutes = function (app) {
  app.route('/restaurants/:restaurantId/tumodelos')
    .get(
      checkEntityExists(Restaurant, 'restaurantId'),
      TuModeloController.index
    )
    .post(
      isLoggedIn,
      hasRole('customer'), // o 'owner'
      checkEntityExists(Restaurant, 'restaurantId'),
      Mid.checkUserHasOrders,
      Mid.checkNotDuplicated,
      TuModeloValidation.create,
      handleValidation,
      TuModeloController.create
    )

  app.route('/restaurants/:restaurantId/tumodelos/:tuModeloId')
    .put(
      isLoggedIn,
      hasRole('customer'),
      checkEntityExists(Restaurant, 'restaurantId'),
      checkEntityExists(TuModelo, 'tuModeloId'),
      Mid.checkOwnership,
      TuModeloValidation.update,
      handleValidation,
      TuModeloController.update
    )
    .delete(
      isLoggedIn,
      hasRole('customer'),
      checkEntityExists(Restaurant, 'restaurantId'),
      checkEntityExists(TuModelo, 'tuModeloId'),
      Mid.checkOwnership,
      Mid.checkHasNoOrders, // Solo si te piden comprobar que no tiene hijos
      TuModeloController.destroy
    )
}
export default loadRoutes
```

---

8. CONSULTAS RÁPIDAS SEQUELIZE (Extra)

Por si necesitas buscar datos a mano dentro de un Middleware o Controlador.

```
// Buscar TODOS los que cumplan una condición múltiple
const items = await Modelo.findAll({
  where: { status: 'pending', userId: req.user.id }
})

// Buscar UNO SOLO (Devuelve objeto o null)
const item = await Modelo.findOne({
  where: { userId: req.user.id, restaurantId: req.params.restaurantId }
})

// Buscar por CLAVE PRIMARIA (El ID)
const itemById = await Modelo.findByPk(req.params.id)

// CONTAR cuántos registros cumplen una condición (Ideal para validaciones)
const total = await Modelo.count({
  where: { userId: req.user.id }
})
```

---

9. EXPANSIÓN NIVEL EXPERTO: Filtros, Tiempos y Validaciones Complejas

A. VALIDACIONES AVANZADAS (Formato de Tiempo y Comparar Campos)

Si te piden validar horas (ej. `08:30:00`) o que un campo sea mayor que otro (ej. `endTime` > `startTime`), usa `.custom()` con funciones externas. **REGLA DE ORO:** Para comparar dos campos del body, extrae el segundo desde `req.body` dentro de la función custom.

```
// 1. Validar formato de hora (HH:mm:ss)
const validateTimeFormat = (value) => {
  const timeRegex = /^([01]\d|2[0-3]):([0-5]\d):([0-5]\d)$/
  if (!timeRegex.test(value)) {
    throw new Error('Time must be in HH:mm:ss format')
  }
  return true
}

// 2. Comparar dos fechas/horas (endTime debe ser mayor que startTime)
const validateEndTimeAfterStartTime = (endTime, { req }) => {
  if (!endTime || !req.body.startTime) return true // Si falta alguno, dejamos que pase (lo p
  if (endTime <= req.body.startTime) {
    throw new Error('End time must be after start time')
  }
  return true
}

// 3. Aplicarlo en el array de validación
const create = [
  check('startTime').exists().custom(validateTimeFormat),
```

```
  check('endTime').exists().custom(validateTimeFormat).custom(validateEndTimeAfterStartTime)
]
```

B. VALIDACIONES CONTRA LA BASE DE DATOS (Promise.reject)

Si necesitas validar que un ID foráneo existe O que pertenece al usuario/restaurante correcto **durante la validación** (antes de llegar al controlador), usa `Promise.reject`. **REGLA DE ORO:** Si el campo es opcional, la primera línea debe ser `if (!value) return Promise.resolve()` para no crashear.

```
const checkScheduleBelongsToRestaurant = async (value, { req }) => {
  if (!value) return Promise.resolve() // Si es opcional y no viene, todo OK

  try {
    // 1. Comprobar que la entidad existe
    const schedule = await Schedule.findByPk(value)
    if (!schedule) {
      return Promise.reject(new Error('The schedule does not exist.'))
    }

    // 2. Comprobar que pertenece al restaurante de la ruta
    if (schedule.restaurantId !== parseInt(req.body.restaurantId)) {
      return Promise.reject(new Error('The schedule does not belong to this restaurant.'))
    }

    return Promise.resolve() // Todo correcto
  } catch (err) {
    return Promise.reject(new Error(err))
  }
}

// Aplicación (Opcionalidad estricta):
check('scheduleId').optional({ nullable: true, checkFalsy: true }).isInt({ min: 1 }).toInt().
```

C. RELACIONES OPCIONALES (Un Modelo puede o no tener otro)

A diferencia de las relaciones obligatorias, si un producto puede tener horario o ser nulo, se configura así:

En la Migración y en el Modelo (Product.js):

```
// Dentro del Product.init(...)
scheduleId: {
    type: DataTypes.INTEGER,
    allowNull: true, // <-- CRÍTICO: Permitir nulos
}
```

En las Asociaciones:

```
// En Product.js (El hijo que guarda el ID)
Product.belongsTo(models.Schedule, { foreignKey: 'scheduleId', as: 'schedule' })

// En Schedule.js (El padre)
Schedule.hasMany(models.Product, { foreignKey: 'scheduleId', as: 'products' })
```

D. EL TRUCO DEL FILTRADO EN MEMORIA (Evitar el Error 404 de Sequelize)

Cuando te piden devolver un Restaurante pero filtrando sus productos (ej. "solo los activos"), **NO** uses `where` dentro del `include` del modelo hijo. Si no hay productos activos, Sequelize oculta el restaurante entero y devuelve `null`. **REGLA DE ORO:** Trae el restaurante con TODO, y filtra el array `.products` en JavaScript antes del `res.json()`.

```
const showWithActiveProducts = async function (req, res) {
  try {
    // 1. Sequelize: Traemos el Restaurante y TODOS sus productos
    const restaurant = await Restaurant.findByPk(req.params.restaurantId, {
      include: {
        model: Product,
        as: 'products',
        include: { model: Schedule, as: 'schedule' }
      }
    })

    if (!restaurant) return res.status(404).send('Restaurant not found.')

    const currentTime = new Date().toTimeString().split(' ')[0]

    // 2. JavaScript: Filtramos el array a mano
    restaurant.dataValues.products = restaurant.products.filter(product => {
      if (!product.schedule) return false // Descarta los que no cumplen la condición
      return product.schedule.startTime <= currentTime && product.schedule.endTime >= current
    })

    res.json(restaurant)
  } catch (err) {
    res.status(500).send(err.message)
  }
}
```

E. AÑADIR RUTAS PERSONALIZADAS DE SOLO LECTURA

Si te inventas una ruta nueva (como `/restaurants/:restaurantId/activos`), debes colocarla con cuidado para aprovechar los middlewares base. **REGLA DE ORO:** Las rutas personalizadas suelen ser `.get()` y solo requieren verificar que el padre exista.

```
// En routes/RestaurantRoutes.js
app.route('/restaurants/:restaurantId/showWithActiveProducts')
  .get(
```

```
  checkEntityExists(Restaurant, 'restaurantId'), // Reutilizamos el middleware base
  RestaurantController.showWithActiveProducts    // Llamamos a nuestra función
)
```